KNOWLEDGE INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai

Kakapalayam (Po), Salem – 637 504

RECORD NOTEBOOK

REG. No.

Certified that this is the bonafide record of work done by Selvam/Selvi ..... of the **V** Semester **B.Tech. Information Technology** Branch during the year **2026 - 27** (Odd Semester) in the **BE23CS410 – C# and DOT NET** Laboratory.

Staff – In charge **Head of the Department**

Submitted for the Model Practical Examination on …….

Internal Examiner

CONTENTS

S. NO.	DATE	NAME OF THE EXPERIMENT	PG. NO.	MARKS	SIGNATURE
1(a)		Write a program to C# to find the smallest single digit factor for a given value.			
1(b)		Write a program in C# to print a number if it is prime; otherwise display the largest factor of that number.			
1(c)		Write a program in C# to find the magnitude of a number.			
2		Write a C# programs to demonstrate the concepts of Constructors and Inheritance.			
3		Write a C# programs to demonstrate the concepts of Partial classes and Extension methods.			
4(a)		Write a C# programs to demonstrate the concepts of Label, Text Box and Button controls.			
4(b)		Write a C# programs to demonstrate the concepts of Combo Box and List Box controls.			
5		Create a Windows application in C# having two text boxes and three buttons named as factorial, prime, factorial series. When you click any button the resultant value will be displayed on the second textbox.			
6		Create a ADO.NET applications in C# to demonstrate the Data Reader, Data Set, Data Adapter and Data View Objects.			
7		Design an ASP.NET Webpage to demonstrate the Label, Button and Textbox, Dropdown list and ListBox controls.			
8		Write a C# program to implement Assemblies			
9		Create a Simple WPF Calculator Using XAML.			
10		Write a C# program that processes a large list of integers sequentially. The operations are as follows: • Filter out the even numbers. • Square the remaining odd numbers			
TOTAL					
AVERAGE					

Department of Information Technology

Vision:

To equip students with cutting-edge information technologies and nurture them into socially responsible and globally recognized Professionals.

Mission:

1. To deliver reliable education with innovative techniques, software updates and programming languages to the students.
2. To cultivate innovation and entrepreneurial spirit among students, enabling them to cater to multidisciplinary sectors.
3. To impart skills to meet the growing demands in academics and industry.
4. To shape the students as sustainable professionals with resilient ethics and society consciousness.

Program Educational Objectives (PEOs):

Graduates will be able to

PEO1: Have a successful career as an IT professional with analytical and critical thinking to meet the diversified requirements of industry, academia and research.

PEO2: Acquire leadership qualities with technical skills and entrepreneurship skills to solve complex engineering and social problems with ethics and environmental responsibility

PEO3: Pursue lifelong learning, and involve in applied research to design optimal solutions

Programme Specific Outcome (PSOs):

To ensure graduates

PSO1: Have proficiency in programming skills to design, develop and apply appropriate techniques, to solve complex engineering problems.

PSO2: Have knowledge to provide technological solutions for automation.

PSO3: Have the skill to secure and manage large volume of data.

Program Outcomes (POs)		Engineering Graduates will be able to
PO1	**Engineering Knowledge**	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem Analysis**	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/ Development of Solutions**	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct Investigations of Complex Problems**	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	**Modern Tool Usage**	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
PO6	**The Engineer and Society**	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	**Environment and Sustainability**	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	**Ethics**	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	**Individual and Team Work**	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication**	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project Management and Finance**	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-Long Learning**	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

LIST OF EQUIPMENTS

Hardware Requirements:

Desktop Computers or Laptops (1 per student- 30 Nos)

Software Requirements:

1. Windows Operating System (Windows 10 or later)
2. Visual Studio IDE (Community Edition is free and sufficient)
    - Ensure .NET Desktop Development workload is installed
    - Windows Forms App (.NET Framework or .NET Core) support
3. .NET SDK (Latest stable version; e.g., .NET 6 or .NET 8)
4. Basic Text Editor (e.g., Notepad++, VS Code – optional)

	Course Outcomes: Upon completion of this course, the students will be able to:	POs/PSOs	BTL	KC	Lab Sessions (Hrs)
CO1	Impart understanding of basic programming concepts in C# language.	PO1, PO2, PO3, PO5, PO12, PSO1	L2	P	3
CO2	Develop real time applications using objectoriented concepts with efficient memory management and exceptional handling.	PO1, PO2, PO3, PO5, PO12, PSO1	L3	P	3
CO3	Design and develop windows and web based applications using C# along with ActiveX Data Object (ADO.Net) concepts.	PO1, PO2, PO3, PO5, PO9, PO10, PO12, PSO1, PSO2	L3	P	8
CO4	Apply advanced .NET concepts and eXtensible Application Markup Language (XAML) to develop dynamic and interactive applications.	PO1, PO2, PO3, PO5, PO9, PO10, PO12, PSO1, PSO2	L3	P	8
CO5	Design and develop interactive desktop applications using Windows Presentation Foundation (WPF) and build service oriented solutions using Windows Communication Foundation (WCF).	PO1, PO2, PO3, PO5, PO9, PO10, PO11, PO12, PSO1, PSO2	L3	P	8
				Total Hrs	**30**

BTL: Bloom Taxonomy Level (L1: Remember, L2: Understand, L3; Apply, L4: Analyze, L5: Evaluate, L6: Create)
KC-Knowledge component (F: Factual, C: Conceptual, P: Procedural, MC: Meta-Cognitive)

Mapping of COs with POs and PSOs

COs	POs												PSOs		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	2	1		2							1	2		
CO2	3	3	3		2							2	3		
CO3	3	3	3		3				2	1		1	3	3	
CO4	3	3	3		3				2	2		2	2	2	
CO5	3	2	3		3				2	2	2	2	2	3	
Avg.	3.0	2.6	2.6		2.6				2.0	1.6	2.0	1.6	2.4	2.2	
1–Low, 2 –Medium, 3–High.															

Ex.No: 1a Date:	Smallest single digit factor

AIM:

To write a C# program to find and display the smallest single-digit factor (1 to 9) of a given number.

ALGORITHM:

1. Start
2. Read an integer value from the user.
3. Loop through numbers from 1 to 9.
4. For each number, check if it divides the given number exactly (i.e., number % i == 0).
5. If true, display it as the smallest single-digit factor and exit the loop.
6. If none of the single-digit numbers divide it, display an appropriate message.
7. Stop

PROGRAM:

```
using System;
class Program
{
   static void Main()
   {
      Console.Write("Enter a number: ");
      int n = int.Parse(Console.ReadLine());

      int factor = -1;

      for (int i = 2; i <= 9; i++)
      {
         if (n % i == 0)
         {
            factor = i;
            break;
         }
      }
```

```
        if (factor != -1)
            Console.WriteLine("Smallest single digit factor: " + factor);
        else
            Console.WriteLine("No single digit factor found.");
    }
}
```

SAMPLE OUTPUT:

Enter a number: 18

Smallest single digit factor: 2

DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, C# program to find and display the smallest single-digit factor (1 to 9) for a given number has been executed and verified successfully.

Ex.No: 1b Date:	Prime Number

AIM:

To write a C# program to printif the given number is prime. If it is not a prime number, then display the largest factor of the number.

ALGORITHM:

1. Start
2. Read an integer number n from the user.
3. If n < 2, it is not a prime.
4. If any factor divides n, mark it as not prime.
5. If it is prime, display: “n is a prime number.”
6. If not prime:
    - Loop from n/2 down to 1.
    - Find the **largest factor** that divides n evenly.
7. Display the largest factor.
8. Stop.

PROGRAM:

```
using System;
class Program
{
   static void Main()
   {
      Console.Write("Enter a number: ");
      int n = int.Parse(Console.ReadLine());

      bool isPrime = true;

      if (n < 2)
         isPrime = false;

      for (int i = 2; i <= Math.Sqrt(n); i++)
      {
         if (n % i == 0)
```

```
            {
                isPrime = false;
                break;
            }
        }

        if (isPrime)
        {
            Console.WriteLine(n + " is a prime number.");
        }
        else
        {
            int largestFactor = 1;

            for (int i = 1; i < n; i++)
            {
                if (n % i == 0)
                    largestFactor = i;
            }

            Console.WriteLine("Largest factor: " + largestFactor);
        }
    }
}
```

SAMPLE OUTPUT:

Enter a number: 13

13 is a prime number.

Enter a number: 20

Largest factor : 10

DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, the C# program to print the prime number and displays the largest factor if the given number is not prime has been executed and verified successfully.

Ex.No: 1c Date:	Magnitude of a number

AIM:

To write a C# program to find the magnitude (i.e. absolute value) of a given number using built-in functionality.

ALGORITHM:

1. Start
2. Prompt the user to enter an integer number.
3. Read the number and store it in a variable.
4. Use Math.Abs() method to find the absolute value (magnitude).
5. Display the result.
6. Stop.

PROGRAM:

```
using System;
class Program
{
   static void Main()
   {
      Console.Write("Enter a number: ");
      double n = double.Parse(Console.ReadLine());
      double magnitude = Math.Abs(n);
      Console.WriteLine("Magnitude of the number: " + magnitude);
   }
}
```

SAMPLE OUTPUT:

Enter a number: 426

Magnitude of the number : 426

DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, the C# program to find the magnitude of a given number has been executed and verified successfully.

Ex.No: 2 Date:	Constructors and Inheritance

AIM:

To write a C# program that demonstrates the concept of constructors (default and parameterized) and inheritance (using base and derived classes).

ALGORITHM:

1. Start
2. Define a base class Person with:
   - Fields: name, age
   - A parameterized constructor to initialize name and age
3. Define a derived class Student inheriting from Person:
   - Add field: studentId
   - A parameterized constructor that calls the base class constructor using base()
4. In the Main() method:
   - Create an object of the Student class
   - Display the student details using a method
5. End

PROGRAM:

```
using System;

class Person
{
   protected string name;
   protected int age;

   public Person(string name, int age)
   {
      this.name = name;
      this.age = age;

      Console.WriteLine("Person constructor called.");
   }
}
class Student : Person
{
   private int studentId;
```

```
    public Student(string name, int age, int studentId)
        : base(name, age)
    {
        this.studentId = studentId;

        Console.WriteLine("Student constructor called.");
    }

    public void DisplayDetails()
    {
        Console.WriteLine("Student Details:");
        Console.WriteLine("Name: " + name);
        Console.WriteLine("Age: " + age);
        Console.WriteLine("Student ID: " + studentId);
    }
}

class Program
{
    static void Main()
    {
        Student student = new Student("John", 20, 1001);

        student.DisplayDetails();
    }
}
```

SAMPLE OUTPUT:

Person constructor called.

Student constructor called.

Student Details:

Name: John

Age: 20

Student ID: 1001

DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, the C# program that demonstrates the concept of constructors and inheritance has been executed and implemented successfully.

Ex.No: 3 Date:	Partial classes and Extension methods

AIM:

To write a C# program to demonstrate the use of partial classes and extension methods.

ALGORITHM:

1. Start
2. Create a class Employee using the partial keyword and split its definition into two parts:
   - One part to define data members
   - Another part to define methods
3. Create a static class EmployeeExtensions with an extension method to display employee details.
4. In the Main() method:
   - Create an object of Employee
   - Call the extension method on that object
5. Stop

PROGRAM:

```
using System;

partial class Employee
{
   public string Name;
   public int EmployeeId;

   public void Welcome()
   {
      Console.WriteLine("Welcome to the Company!");
   }
}

partial class Employee
{
   public void DisplayEmployee()
   {
      Console.WriteLine("Employee Name: " + Name);
      Console.WriteLine("Employee ID: " + EmployeeId);
   }
}
```

```
static class EmployeeExtensions
{
   public static void DisplayDetails(this Employee employee)
   {
      employee.Welcome();
      employee.DisplayEmployee();
   }
}

class Program
{
   static void Main()
   {
      Employee employee = new Employee();

      employee.Name = "Alice";
      employee.EmployeeId = 101;

      employee.DisplayDetails();
   }
}
```


SAMPLE OUTPUT:

Welcome to the Company!

Employee Name: Alice

Employee ID: 101

DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, a C# program to demonstrate the use of partial classes and extension methods has been executed and verified successfully.

Ex.No: 4a Date:	Label, Text Box and Button controls

AIM:

To write a C# program to demonstrate the use of Label, TextBox, and Button controls in Windows Forms.

ALGORITHM:

1. Start-> open Visual Studio
2. Create a new Windows Forms Application.
3. Drag and drop a Label, a TextBox, and a Button on the Form.
4. Set the properties (like Text, Name) of the controls appropriately.
5. Write the Button click event handler to display the entered text from the TextBox using another Label or MessageBox.
6. Stop

PROGRAM:

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Experiment4A
{
   public class Form1 : Form
   {
      Label nameLabel;
      Label organizationLabel;
      Label commentLabel;
      TextBox nameTextBox;
      TextBox organizationTextBox;
      TextBox commentTextBox;
      Button sendButton;

      public Form1()
      {
         Text = "tutorialspont.com";
         Size = new Size(530, 360);
         StartPosition = FormStartPosition.CenterScreen;
         FormBorderStyle = FormBorderStyle.FixedSingle;
         MaximizeBox = false;

         nameLabel = new Label();
```

```
nameLabel.Text = "Name:";
nameLabel.Location = new Point(35, 45);
nameLabel.Size = new Size(120, 25);
nameLabel.Font = new Font("Arial", 11, FontStyle.Bold);

nameTextBox = new TextBox();
nameTextBox.Location = new Point(185, 42);
nameTextBox.Size = new Size(280, 25);

organizationLabel = new Label();
organizationLabel.Text = "Organization:";
organizationLabel.Location = new Point(35, 100);
organizationLabel.Size = new Size(140, 25);
organizationLabel.Font = new Font("Arial", 11, FontStyle.Bold);

organizationTextBox = new TextBox();
organizationTextBox.Location = new Point(185, 97);
organizationTextBox.Size = new Size(280, 25);

commentLabel = new Label();
commentLabel.Text = "Comment:";
commentLabel.Location = new Point(35, 165);
commentLabel.Size = new Size(120, 25);
commentLabel.Font = new Font("Arial", 11, FontStyle.Bold);

commentTextBox = new TextBox();
commentTextBox.Location = new Point(185, 160);
commentTextBox.Size = new Size(280, 35);
commentTextBox.Multiline = true;
commentTextBox.ScrollBars = ScrollBars.Vertical;

sendButton = new Button();
sendButton.Text = "Send Message";
sendButton.Location = new Point(185, 235);
sendButton.Size = new Size(140, 30);

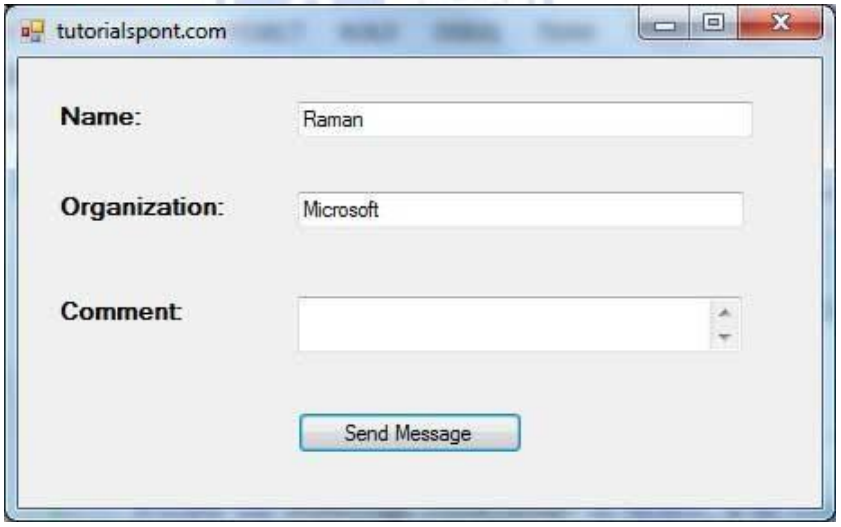
sendButton.Click += SendButton_Click;

Controls.Add(nameLabel);
Controls.Add(nameTextBox);
Controls.Add(organizationLabel);
Controls.Add(organizationTextBox);
Controls.Add(commentLabel);
Controls.Add(commentTextBox);
Controls.Add(sendButton);
```

```
    }

    private void SendButton_Click(object sender, EventArgs e)
    {
      MessageBox.Show(
        "Name: " + nameTextBox.Text +
        "\nOrganization: " + organizationTextBox.Text +
        "\nComment: " + commentTextBox.Text,
        "Message Details"
      );
    }
    [STAThread]
    static void Main()
    {
      Application.EnableVisualStyles();
      Application.Run(new Form1());
    }
  }
}
```

SAMPLE OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, the C# program to demonstrate the use of Label, TextBox, and Button controls was successfully implemented and tested.

Ex.No: 4b Date:	Combo Box and List Box controls

AIM:

To write a C# Windows Forms application that demonstrates the usage of ComboBox and ListBox controls to display and select items.

ALGORITHM:

1. Start
2. Open Visual Studio
3. Create a new Windows Forms App (.NET Framework) project
4. Design the Form by:
   - Adding a ComboBox control.
   - Adding a ListBox control.
   - Adding a Button control.
5. Add items to the ComboBox either:
   - At design time (via Properties window), or
   - At runtime in Form_Load event.
6. Write the Button click event code to:
   - Get the selected item from the ComboBox.
   - Add the selected item to the ListBox.
7. Run the application.
8. Select an item from ComboBox and click the Button to see it added to ListBox.
9. Stop

PROGRAM:

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Experiment4B
{
   public class Form1 : Form
   {
      Label titleLabel;
      ComboBox comboBox;
      ListBox listBox;
      Button addButton;
      Button sortButton;
      Button clearButton;

      public Form1()
      {
         Text = "tutorialspont.com";
```

```
Size = new Size(530, 290);
StartPosition = FormStartPosition.CenterScreen;
FormBorderStyle = FormBorderStyle.FixedSingle;
MaximizeBox = false;

titleLabel = new Label();
titleLabel.Text = "Wish List for 2013:";
titleLabel.Location = new Point(25, 20);
titleLabel.AutoSize = true;

comboBox = new ComboBox();
comboBox.Location = new Point(25, 45);
comboBox.Size = new Size(125, 25);
comboBox.DropDownStyle = ComboBoxStyle.DropDownList;

comboBox.Items.AddRange(new object[]
{
   "Safety",
   "Security",
   "Governance",
   "Good Music",
   "Good Movies",
   "Good Books",
   "Education",
   "Roads",
   "Health",
   "Food for all",
   "Shelter for all",
   "Industrialisation",
   "Peace",
   "Liberty",
   "Freedom of Speech"
});

comboBox.SelectedIndex = 10;

listBox = new ListBox();
listBox.Location = new Point(295, 45);
listBox.Size = new Size(125, 95);

listBox.Items.Add("Good Books");
listBox.Items.Add("Education");
listBox.Items.Add("Food for all");
listBox.Items.Add("Shelter for all");
```

```
    addButton = new Button();
    addButton.Text = ">>";
    addButton.Location = new Point(210, 65);
    addButton.Size = new Size(50, 25);
    addButton.Click += AddButton_Click;

    sortButton = new Button();
    sortButton.Text = "Sort";
    sortButton.Location = new Point(210, 145);
    sortButton.Size = new Size(70, 25);
    sortButton.Click += SortButton_Click;

    clearButton = new Button();
    clearButton.Text = "Clear";
    clearButton.Location = new Point(365, 145);
    clearButton.Size = new Size(70, 25);
    clearButton.Click += ClearButton_Click;

    Controls.Add(titleLabel);
    Controls.Add(comboBox);
    Controls.Add(listBox);
    Controls.Add(addButton);
    Controls.Add(sortButton);
    Controls.Add(clearButton);
}

private void AddButton_Click(object sender, EventArgs e)
{
    if (comboBox.SelectedItem != null)
    {
        string selectedItem = comboBox.SelectedItem.ToString();

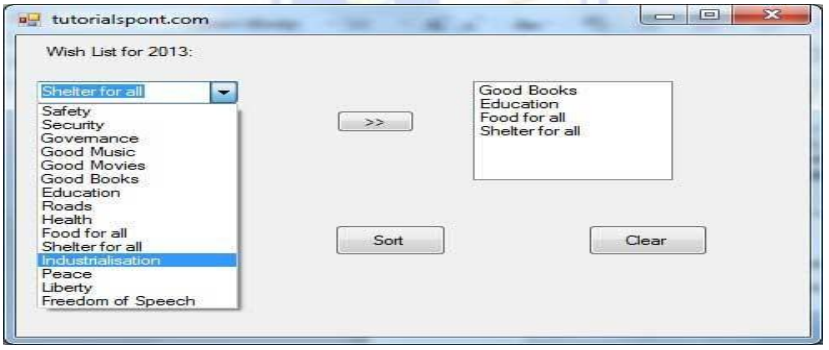
        if (!listBox.Items.Contains(selectedItem))
        {
            listBox.Items.Add(selectedItem);
        }
    }
}

private void SortButton_Click(object sender, EventArgs e)
{
    listBox.Sorted = true;
}
```

```
        private void ClearButton_Click(object sender, EventArgs e)
        {
            listBox.Items.Clear();
        }

        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.Run(new Form1());
        }
    }
}
```

SAMPLE OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, C# Windows Forms application that demonstrates the usage of ComboBox and ListBox controls to display and select items.

Ex.No: 5 Date:	Windows Forms Application

AIM:

To develop a Windows Forms Application in C# to display the prime number, factorial and factorial series of a given number.

ALGORITHM:

1. Start the application.
2. Create a Windows Form with:
    - TextBox1: For input (user enters a number).
    - TextBox2: For output (result displayed here).
    - Button1: Labeled “Factorial”.
    - Button2: Labeled “Prime”.
    - Button3: Labeled “Factorial Series”.
3. When the Factorial button is clicked:
    - Read number from TextBox1.
    - Calculate factorial of the number.
    - Display result in TextBox2.
4. When the Prime button is clicked:
    - Read number from TextBox1.
    - Check if number is prime.
    - Display result in TextBox2.
5. When the Factorial Series button is clicked:
    - Read number from TextBox1.
    - Display factorials from 1 to n in TextBox2.
6. Stop

PROGRAM:

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Experiment5
{
   public class Form1 : Form
   {
```

```
TextBox textBox1;
TextBox textBox2;
Button button1;
Button button2;
Button button3;

public Form1()
{
   Text = "Factorial, Prime, Factorial Series";
   Size = new Size(550, 400);
   StartPosition = FormStartPosition.CenterScreen;

   textBox1 = new TextBox();
   textBox1.Location = new Point(130, 40);
   textBox1.Size = new Size(260, 25);

   textBox2 = new TextBox();
   textBox2.Location = new Point(130, 85);
   textBox2.Size = new Size(260, 135);
   textBox2.Multiline = true;
   textBox2.ScrollBars = ScrollBars.Vertical;

   button1 = new Button();
   button1.Text = "Factorial";
   button1.Location = new Point(35, 250);
   button1.Size = new Size(140, 40);
   button1.Click += Factorial_Click;

   button2 = new Button();
   button2.Text = "Prime";
   button2.Location = new Point(190, 250);
   button2.Size = new Size(140, 40);
   button2.Click += Prime_Click;

   button3 = new Button();
   button3.Text = "Factorial Series";
   button3.Location = new Point(345, 250);
   button3.Size = new Size(140, 40);
   button3.Click += FactorialSeries_Click;

   Controls.Add(textBox1);
   Controls.Add(textBox2);
   Controls.Add(button1);
   Controls.Add(button2);
   Controls.Add(button3);
```

```
}

private void Factorial_Click(object sender, EventArgs e)
{
   int n = int.Parse(textBox1.Text);
   long factorial = 1;

   for (int i = 1; i <= n; i++)
   {
      factorial *= i;
   }

   textBox2.Text = n + "! = " + factorial;
}

private void Prime_Click(object sender, EventArgs e)
{
   int n = int.Parse(textBox1.Text);
   bool isPrime = n >= 2;

   for (int i = 2; i <= n / i; i++)
   {
      if (n % i == 0)
      {
         isPrime = false;
         break;
      }
   }

   if (isPrime)
      textBox2.Text = n + " is a prime number.";
   else
      textBox2.Text = n + " is not a prime number.";
}

private void FactorialSeries_Click(object sender, EventArgs e)
{
   int n = int.Parse(textBox1.Text);
   long factorial = 1;
   string result = "Factorial Series:\r\n";

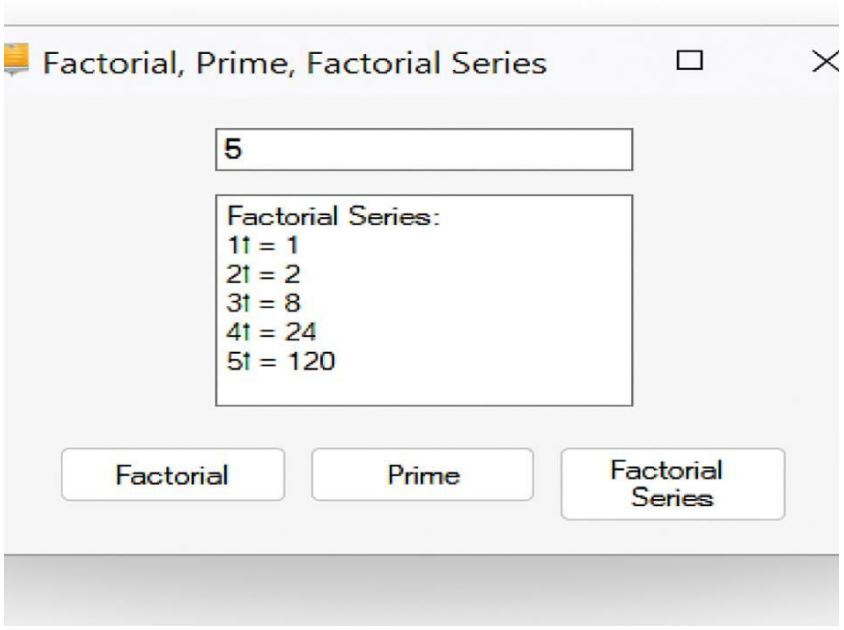
   for (int i = 1; i <= n; i++)
   {
      factorial *= i;
      result += i + "! = " + factorial + "\r\n";
```

```
        }

        textBox2.Text = result;
    }

    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.Run(new Form1());
    }
  }
}
```

SAMPLE OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, Windows Forms Application in C# to display the prime number, factorial and factorial series of a given number has been developed and executed successfully.

Ex.No: 6 Date:	ADO.NET applications

AIM:

To develop a ADO.NET applicationsto demonstrateDataReader**,** DataSet**,** DataAdapter and DataView objects to efficiently retrieve, manipulate, and display student marks data.

PROCEDURE:

1. Setup Your Development Environment

- Install Visual Studio (Community Edition is fine).
- Create a new **Windows Forms App (.NET Framework)** project.

2. Create the Database and Table

- Create a database (e.g., StudentDB) using SQL Server or any preferred database.
- Create a table Students with columns such as:
  - StudentID (int, Primary Key)
  - Name (varchar)
  - Mark1 (int)
  - Mark2 (int)

3. Design the Windows Form

- Add controls to the form for:
  - Displaying student data (e.g., DataGridView).
  - Buttons for loading data, calculating total/average marks.
  - TextBoxes or Labels for showing calculated results.

4. Add ADO.NET Connection to the Database

- Add the necessary namespace references in your code:
- Define your connection string to connect to the database.

5. Use SqlDataReader to Fetch Student Data

- Write a method to fetch student data using SqlDataReader (fast, forward-only):

6. Use DataAdapter and DataSet for Disconnected Data Access

- Use SqlDataAdapter to fill a DataSet with student data:
- Bind the DataSet to controls like DataGridView to display data:

7. Manipulate Data using DataView

- Create a DataView from the DataSet for filtering or sorting:

8. Calculate Total and Average Marks

- You can either:
  - Calculate on-the-fly while reading data.
  - Add computed columns to the DataSet.

Example on-the-fly:

```
inttotalMarks = (int)row["Mark1"] + (int)row["Mark2"] + (int)row["Mark3"];
doubleaverageMarks = totalMarks / 3.0;
```

9. Update Records using DataAdapter

- Use the DataAdapter to update changes back to the database:

  ```
  SqlCommandBuilder builder = newSqlCommandBuilder(adapter);
  adapter.Update(ds, "Students");
  ```

10. Run and Test the Application

- Test loading data with DataReader.
- Test displaying and manipulating data with DataSet and DataView.
- Test updating marks and saving changes back to the database.

PROGRAM:

SQL Server code

```
IF DB_ID('StudentDB') IS NULL
   CREATE DATABASE StudentDB;
GO

USE StudentDB;
GO

IF OBJECT_ID('dbo.Students', 'U') IS NULL
BEGIN
   CREATE TABLE Students
   (
      StudentID INT PRIMARY KEY,
      Name VARCHAR(100),
      Mark1 INT,
      Mark2 INT,
      Mark3 INT
   );

   INSERT INTO Students
      (StudentID, Name, Mark1, Mark2, Mark3)
   VALUES
      (1001, 'John', 80, 85, 90),
      (1002, 'Alice', 75, 80, 85),
      (1003, 'David', 90, 88, 95);
END;
GO

SELECT * FROM Students;
GO
```

C# code - Form1.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Windows.Forms;

namespace Experiment6
{
   public class Form1 : Form
   {
      DataGridView dataGridView;
      Button readerButton;
      Button dataSetButton;
      Button viewButton;
      Button updateButton;
      Label resultLabel;

      SqlDataAdapter adapter;
      SqlCommandBuilder commandBuilder;
      DataSet dataSet;

      string connectionString =
         @"Data Source=.\SQLEXPRESS;Initial Catalog=StudentDB;Integrated Security=True";
      public Form1()
      {
         Text = "ADO.NET Student Marks";
         Width = 950;
         Height = 600;
         StartPosition = FormStartPosition.CenterScreen;

         dataGridView = new DataGridView();
         dataGridView.Location = new Point(30, 30);
         dataGridView.Size = new Size(870, 300);
         dataGridView.AutoSizeColumnsMode =
            DataGridViewAutoSizeColumnsMode.Fill;
         dataGridView.AllowUserToAddRows = false;
         dataGridView.ReadOnly = false;

         readerButton = new Button();
         readerButton.Text = "DataReader";
         readerButton.Location = new Point(30, 350);
         readerButton.Size = new Size(110, 35);
         readerButton.Click += ReaderButton_Click;
```

```
    dataSetButton = new Button();
    dataSetButton.Text = "DataSet + Adapter";
    dataSetButton.Location = new Point(155, 350);
    dataSetButton.Size = new Size(140, 35);
    dataSetButton.Click += DataSetButton_Click;

    viewButton = new Button();
    viewButton.Text = "DataView";
    viewButton.Location = new Point(310, 350);
    viewButton.Size = new Size(100, 35);
    viewButton.Click += ViewButton_Click;

    updateButton = new Button();
    updateButton.Text = "Update";
    updateButton.Location = new Point(425, 350);
    updateButton.Size = new Size(100, 35);
    updateButton.Click += UpdateButton_Click;

    resultLabel = new Label();
    resultLabel.Location = new Point(30, 410);
    resultLabel.AutoSize = true;
    resultLabel.Font = new Font("Arial", 10);

    Controls.Add(dataGridView);
    Controls.Add(readerButton);
    Controls.Add(dataSetButton);
    Controls.Add(viewButton);
    Controls.Add(updateButton);
    Controls.Add(resultLabel);
}
private void ReaderButton_Click(object sender, EventArgs e)
{
    try
    {
        using (SqlConnection connection =
              new SqlConnection(connectionString))
        {
            connection.Open();
            string query = "SELECT * FROM Students";
            SqlCommand command =
                new SqlCommand(query, connection);
            using (SqlDataReader reader =
                  command.ExecuteReader())
            {
                DataTable table = new DataTable();
```

```
                table.Load(reader);
                dataGridView.DataSource = table;
            }
        }
        resultLabel.Text = "Records loaded using DataReader.";
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Database Error");
    }
}
private void DataSetButton_Click(object sender, EventArgs e)
{
    try
    {
        adapter = new SqlDataAdapter(
            "SELECT StudentID, Name, Mark1, Mark2, Mark3 FROM Students",
            connectionString);
        commandBuilder = new SqlCommandBuilder(adapter);
        dataSet = new DataSet();
        adapter.Fill(dataSet, "Students");
        DataTable table = dataSet.Tables["Students"];

        table.Columns.Add("Total", typeof(int));
        table.Columns["Total"].Expression =
            "Mark1 + Mark2 + Mark3";

        table.Columns.Add("Average", typeof(double));
        table.Columns["Average"].Expression =
            "(Mark1 + Mark2 + Mark3) / 3.0";

        dataGridView.DataSource = table;

        resultLabel.Text =
            "DataSet loaded. Total and Average calculated for all students.";
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Database Error");
    }
}

private void ViewButton_Click(object sender, EventArgs e)
{
    if (dataSet == null)
```

```
    {
        MessageBox.Show("Click DataSet + Adapter first.");
        return;
    }

    DataView view =
        new DataView(dataSet.Tables["Students"]);

    view.Sort = "Mark1 DESC";

    dataGridView.DataSource = view;

    resultLabel.Text =
        "Students sorted by Mark1 in descending order.";
}

private void UpdateButton_Click(object sender, EventArgs e)
{
    if (dataSet == null || adapter == null)
    {
        MessageBox.Show("Click DataSet + Adapter first.");
        return;
    }
    try
    {
        dataGridView.EndEdit();

        BindingContext[dataGridView.DataSource]
            .EndCurrentEdit();

        adapter.Update(dataSet, "Students");

        MessageBox.Show(
            "Database updated successfully.",
            "Update"
        );
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Update failed: " + ex.Message,
            "Database Error"
        );
    }
}
```

```
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.Run(new Form1());
        }
    }
}
```

OUTPUT:

StudentID	Name	Mark1	Mark2	Mark3	Total	Average
1001	John	80	85	90	255	85
1002	Alice	75	80	85	240	80
1003	David	90	88	95	273	91

DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, ADO.NET application to demonstrate DataReader**,** DataSet**,** DataAdapter and DataView objects has been executed and verified successfully.

Ex.No: 7a Date:	ASP.NET Webpage with web controls

AIM:

To build a user registration form using ASP.NET Web Forms with various web controls like TextBox, DropDownList, RadioButtonList, CheckBox, Button, and Label.

PROCEDURE:

- Create a new ASP.NET Web Forms project in Visual Studio.
- Add a new Web Form (e.g., Register.aspx).

Design the form in Register.aspx

Add the following web controls inside a <form runat="server"> tag:

Field	Control Type	Purpose
Full Name	TextBox	Input user's full name
Email	TextBox	Input user's email
Password	TextBox (TextMode=Password)	Input password
Gender	RadioButtonList	Select gender (Male, Female)
Country	DropDownList	Select country from list
Accept Terms	CheckBox	Confirm acceptance of terms
Submit Button	Button	Submit the form
Result Display	Label	Show success or error message

Add **RegularExpressionValidator** controls to validate email format and password strength.

PROGRAM:

```
using System;
namespace Experiment7
{
   public partial class Register : System.Web.UI.Page
   {
      protected void btnSubmit_Click(object sender, EventArgs e)
      {
         Page.Validate();

         if (!Page.IsValid)
         {
            lblResult.Text = "Please correct the errors.";
            lblResult.ForeColor =
               System.Drawing.Color.Red;
            return;
         }
         if (rblGender.SelectedIndex == -1)
         {
            lblResult.Text = "Please select your gender.";
            lblResult.ForeColor =
               System.Drawing.Color.Red;
            return;
         }
         if (ddlCountry.SelectedValue == "")
         {
            lblResult.Text = "Please select your country.";
            lblResult.ForeColor =
               System.Drawing.Color.Red;
            return;
         }
         if (!chkTerms.Checked)
         {
            lblResult.Text =
               "Please accept the terms and conditions.";
            lblResult.ForeColor =
               System.Drawing.Color.Red;
            return;
         }
         lblResult.Text =
            "Registration Successful!<br/>" +
            "Name: " +
            Server.HtmlEncode(txtName.Text) +
```

```
            "<br/>Email: " +
            Server.HtmlEncode(txtEmail.Text) +
            "<br/>Gender: " +
            Server.HtmlEncode(rblGender.SelectedValue) +
            "<br/>Country: " +
            Server.HtmlEncode(ddlCountry.SelectedValue);
        lblResult.ForeColor =
            System.Drawing.Color.Green;
    }
  }
}
```

OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, a user registration form using ASP.NET Web Forms with various web controls like TextBox, DropDownList, RadioButtonList, CheckBox, Button, and Label has been executed and verified successfully.

Ex.No: 7 (b)	ASP.NET Webpage to Save Form Data to a Database

AIM:

To enhance the ASP.NET Web Forms user registration form by saving the submitted user data into a database, ensuring data persistence beyond the session.

PROCEDURE:

1. **Set up the Database**

- Create a database (e.g., UserDB).
- Create a table Users with columns:
  - Id (int, Primary Key, Identity)
  - FullName (nvarchar(100))
  - Email (nvarchar(100))
  - Password (nvarchar(100))
  - Gender (nvarchar(10))
  - Country (nvarchar(50))

Example SQL Server script:

```
CREATETABLE Users (
   Id INTPRIMARY KEY IDENTITY(1,1),
FullNameNVARCHAR(100),
   Email NVARCHAR(100),
   Password NVARCHAR(100),
   Gender NVARCHAR(10),
   Country NVARCHAR(50));
```

2. **Add Connection String**

- In Web.config, add your database connection string inside <connectionStrings> section:

```
xml
CopyEdit
<connectionStrings>
```

<addname="UserDBConnectionString"connectionString="Data Source=.\SQLEXPRESS;Initial Catalog=UserDB;Integrated Security=True"providerName="System.Data.SqlClient" />
</connectionStrings>

3. **Modify the code-behind (Register.aspx.cs)**

- Add using System.Data.SqlClient;
- Get the connection string from config.
- Creates a new SQL connection object conn using the connection string.
- The using statement ensures the connection is properly closed and disposed of automatically after the block completes, even if an error occurs.
- Create a command object and add parameters
- On button click, after validation, insert data into the database

PROGRAM:

SQL Server code

```
IF DB_ID('UserDB') IS NULL
   CREATE DATABASE UserDB;
GO

USE UserDB;
GO

IF OBJECT_ID('dbo.Users', 'U') IS NULL
BEGIN
   CREATE TABLE Users
   (
      Id INT PRIMARY KEY IDENTITY(1,1),
      FullName NVARCHAR(100),
      Email NVARCHAR(100),
      Password NVARCHAR(100),
      Gender NVARCHAR(10),
      Country NVARCHAR(50)
   );
END;
GO
```

Register.aspx.cs

```
using System;
using System.Configuration;
using System.Data.SqlClient;

namespace Experiment7
{
   public partial class Register : System.Web.UI.Page
   {
      protected void btnSubmit_Click(object sender, EventArgs e)
      {
         Page.Validate();

         if (!Page.IsValid)
         {
            lblResult.Text = "Please correct the validation errors.";
            return;
         }

         if (rblGender.SelectedIndex == -1)
         {
            lblResult.Text = "Please select gender.";
            return;
         }

         if (ddlCountry.SelectedValue == "")
         {
            lblResult.Text = "Please select country.";
            return;
         }

         if (!chkTerms.Checked)
         {
            lblResult.Text = "Please accept the terms.";
            return;
         }

         string connectionString =
            ConfigurationManager.ConnectionStrings[
               "UserDBConnectionString"].ConnectionString;

         try
         {
            using (SqlConnection conn =
```

```
            new SqlConnection(connectionString))
        {
            string query =
                "INSERT INTO Users " +
                "(FullName, Email, Password, Gender, Country) " +
                "VALUES (@FullName, @Email, @Password, @Gender, @Country)";

            using (SqlCommand cmd =
                new SqlCommand(query, conn))
            {
                cmd.Parameters.AddWithValue(
                    "@FullName", txtName.Text.Trim());

                cmd.Parameters.AddWithValue(
                    "@Email", txtEmail.Text.Trim());

                cmd.Parameters.AddWithValue(
                    "@Password", txtPassword.Text);

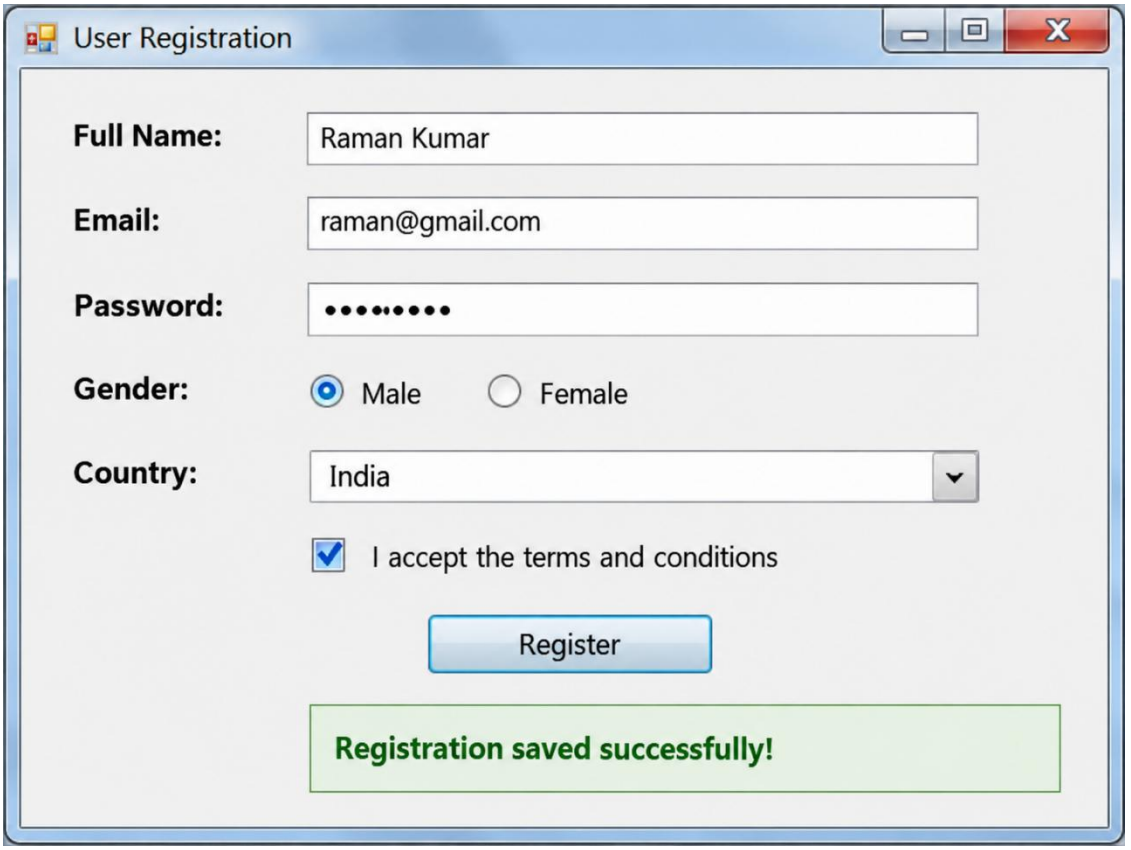
                cmd.Parameters.AddWithValue(
                    "@Gender", rblGender.SelectedValue);

                cmd.Parameters.AddWithValue(
                    "@Country", ddlCountry.SelectedValue);

                conn.Open();
                cmd.ExecuteNonQuery();
            }
        }

        lblResult.Text = "Registration saved successfully!";
    }
    catch (Exception ex)
    {
        lblResult.Text =
            "Error saving registration: " +
            Server.HtmlEncode(ex.Message);
    }
  }
}
```

OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, the ASP.NET Web Forms for user registration has been done and executed successfully.

Ex.No: 8(a) Date:	Assemblies-DLL

AIM:

To build a reusable class library (DLL) assembly in C# and use it in a console application.

PROCEDURE:

Create Class Library Project

- In Visual Studio, create a new **Class Library (.NET Framework)** project called MathLibrary.
- Add a public class Calculator with methods like Add(int a, int b), Subtract(int a, int b).

Build the Class Library

- Build the project to generate MathLibrary.dll.

Create Console Application

- Create a new Console App project called MathApp.
- Add a reference to the MathLibrary.dll you just created.

Use the Assembly

- In Program.cs of the console app, instantiate the Calculator class and call its methods.

Run the Console App

  - Verify output shows correct addition and subtraction results.

PROGRAM:

Calculator.cs

```
namespace MathLibrary
{
   public class Calculator
   {
      public int Add(int a, int b)
      {
         return a + b;
      }
```

```
        public int Subtract(int a, int b)
        {
            return a - b;
        }
    }
}
```

Program.cs

```
using System;
using MathLibrary;

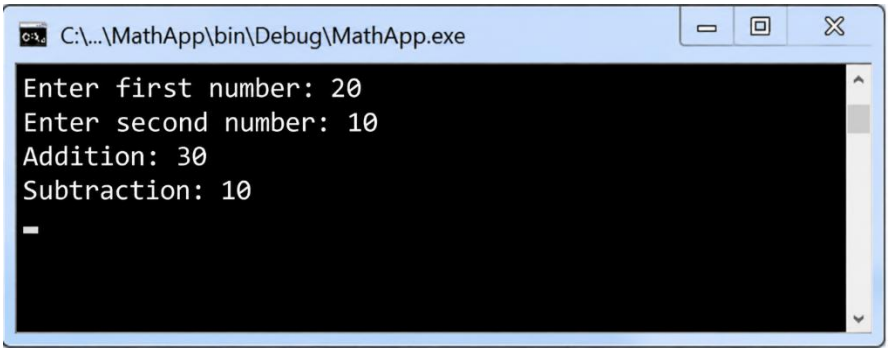
class Program
{
    static void Main()
    {
        Calculator calculator = new Calculator();

        Console.Write("Enter first number: ");
        int a = int.Parse(Console.ReadLine());

        Console.Write("Enter second number: ");
        int b = int.Parse(Console.ReadLine());

        Console.WriteLine("Addition: " + calculator.Add(a, b));
        Console.WriteLine("Subtraction: " + calculator.Subtract(a, b));
    }
}
```

OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, a reusable class library (DLL) assembly in C# has been used in a console application and executed successfully.

Ex.No: 8(b) Date:	Assemblies - Reflection

AIM:

To write a program that loads an existing assembly and reads its metadata like version, name, and public types using Reflection.

PROCEDURE:

1. **Create Console Application**

- Create a new Console App project named AssemblyInfoReader.

2. **Load an Assembly**

- Use Assembly.LoadFrom() to load an external assembly (for example, the MathLibrary.dll from previous exercise.

3. **Read Metadata**

- Display the assembly's full name, version, and list all public types.

4. **Run the program**

- Check that it outputs correct assembly info and lists types like MathLibrary.Calculator.

PROGRAM:

```
using System;
using System.Reflection;

class Program
{
   static void Main()
   {
      string path = @"C:\MathLibrary\bin\Debug\MathLibrary.dll";

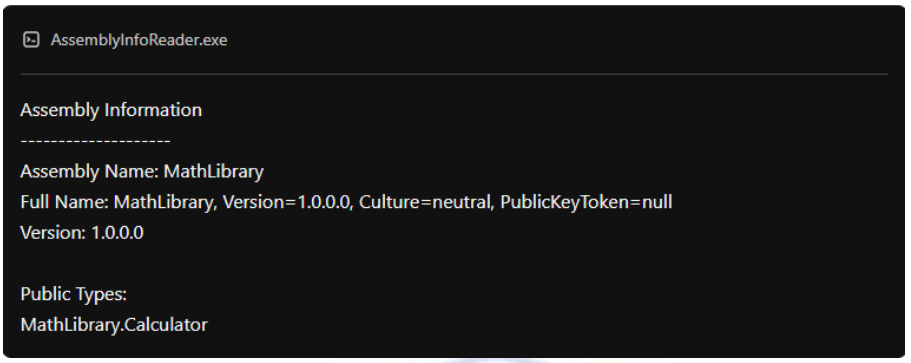
      Assembly assembly = Assembly.LoadFrom(path);

      Console.WriteLine("Assembly Information");
      Console.WriteLine("--------------------");
      Console.WriteLine("Assembly Name: " + assembly.GetName().Name);
      Console.WriteLine("Full Name: " + assembly.FullName);
      Console.WriteLine("Version: " + assembly.GetName().Version);
```

```
        Console.WriteLine("\nPublic Types:");

        foreach (Type type in assembly.GetExportedTypes())
        {
            Console.WriteLine(type.FullName);
        }
    }
}
```

OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus,a program that loads an existing assembly and reads its metadata like version, name, and public types using Reflection has been executed successfully.

Ex.No: 9 Date:	Simple WPF Calculator Using XAML

AIM:

To design and develop a basic calculator using Windows Presentation Foundation (WPF).

PROCEDURE:

1. Create a New WPF Application

1. Open **Visual Studio**.
2. Go to **File > New > Project**.
3. Select **WPF App (.NET Framework)** (or **WPF App (.NET Core)** depending on your setup).
4. Name it WpfCalculator.
5. Click **Create**.

2. Design the Calculator Interface (XAML)

```
<Window x:Class="WpfCalculator.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Calculator"
        Height="400"
        Width="300"
        WindowStartupLocation="CenterScreen">

    <Grid>
        <TextBox Name="txtDisplay"
                 FontSize="24"
                 Margin="10"
                 Height="60"
                 VerticalAlignment="Top"
                 IsReadOnly="True"
                 TextAlignment="Right"
                 VerticalContentAlignment="Center"/>

        <Grid Margin="10,80,10,10">
            <Grid.RowDefinitions>
                <RowDefinition Height="*"/>
                <RowDefinition Height="*"/>
```

```
            <RowDefinition Height="*"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>

        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <Button Content="7" Grid.Row="0" Grid.Column="0" Click="Button_Click"/>
        <Button Content="8" Grid.Row="0" Grid.Column="1" Click="Button_Click"/>
        <Button Content="9" Grid.Row="0" Grid.Column="2" Click="Button_Click"/>
        <Button Content="/" Grid.Row="0" Grid.Column="3" Click="Operator_Click"/>

        <Button Content="4" Grid.Row="1" Grid.Column="0" Click="Button_Click"/>
        <Button Content="5" Grid.Row="1" Grid.Column="1" Click="Button_Click"/>
        <Button Content="6" Grid.Row="1" Grid.Column="2" Click="Button_Click"/>
        <Button Content="*" Grid.Row="1" Grid.Column="3" Click="Operator_Click"/>

        <Button Content="1" Grid.Row="2" Grid.Column="0" Click="Button_Click"/>
        <Button Content="2" Grid.Row="2" Grid.Column="1" Click="Button_Click"/>
        <Button Content="3" Grid.Row="2" Grid.Column="2" Click="Button_Click"/>
        <Button Content="-" Grid.Row="2" Grid.Column="3" Click="Operator_Click"/>

        <Button Content="0" Grid.Row="3" Grid.Column="0" Click="Button_Click"/>
        <Button Content="C" Grid.Row="3" Grid.Column="1" Click="Clear_Click"/>
        <Button Content="=" Grid.Row="3" Grid.Column="2" Click="Equals_Click"/>
        <Button Content="+" Grid.Row="3" Grid.Column="3" Click="Operator_Click"/>
      </Grid>
    </Grid>
</Window>
```

3. Write Code-Behind Logic (MainWindow.xaml.cs)

```
using System;
using System.Windows;
using System.Windows.Controls;

namespace WpfCalculator
{
    public partial class MainWindow : Window
    {
```

```
double firstNumber = 0;
string operation = "";
public MainWindow()
{
    InitializeComponent();
}

private void Button_Click(object sender, RoutedEventArgs e)
{
    Button button = (Button)sender;
    txtDisplay.Text += button.Content.ToString();
}

private void Operator_Click(object sender, RoutedEventArgs e)
{
    Button button = (Button)sender;

    if (txtDisplay.Text == "")
        return;

    firstNumber = double.Parse(txtDisplay.Text);
    operation = button.Content.ToString();
    txtDisplay.Text = "";
}

private void Equals_Click(object sender, RoutedEventArgs e)
{
    if (txtDisplay.Text == "")
        return;

    double secondNumber = double.Parse(txtDisplay.Text);
    double result = 0;

    if (operation == "+")
        result = firstNumber + secondNumber;
    else if (operation == "-")
        result = firstNumber - secondNumber;
    else if (operation == "*")
        result = firstNumber * secondNumber;
    else if (operation == "/")
    {
        if (secondNumber == 0)
        {
            txtDisplay.Text = "Error";
            return;
```

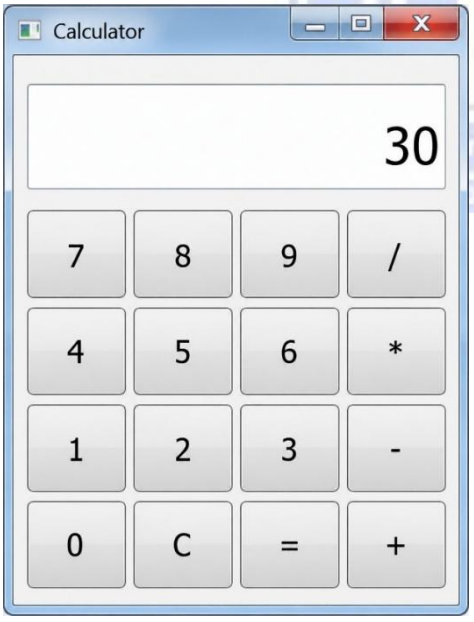
```
            }
            result = firstNumber / secondNumber;
        }
        else
            return;

        txtDisplay.Text = result.ToString();
        operation = "";
    }

    private void Clear_Click(object sender, RoutedEventArgs e)
    {
        txtDisplay.Text = "";
        firstNumber = 0;
        operation = "";
    }
  }
}
```

4. Run the Application

OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, a basic calculator using Windows Presentation Foundation (WPF) has been developed successfully.

Ex.No: 10 Date:	Sequential Processing Using LINQ

AIM:

To write and execute a C# program that filters out even numbers from a list and squares the remaining odd numbers using sequential processing.

PROCEDURE:

- Generate a list of integers from 1 to 1,000,000.
- Use a filter to remove even numbers (i.e., keep numbers where number % 2 != 0).
- Square each of the remaining odd numbers (i.e., number * number).
- Store the processed results in a new list.
- Display the first 10 squared numbers as sample output.
- Print the total number of odd numbers processed.

PROGRAM:

```
using System;
using System.Linq;
using System.Collections.Generic;

class Program
{
   static void Main()
   {
      List<int> numbers = Enumerable.Range(1, 1000000).ToList();

      List<long> squaredOddNumbers = numbers
         .Where(number => number % 2 != 0)
         .Select(number => (long)number * number)
         .ToList();

      Console.WriteLine("First 10 squared odd numbers:");

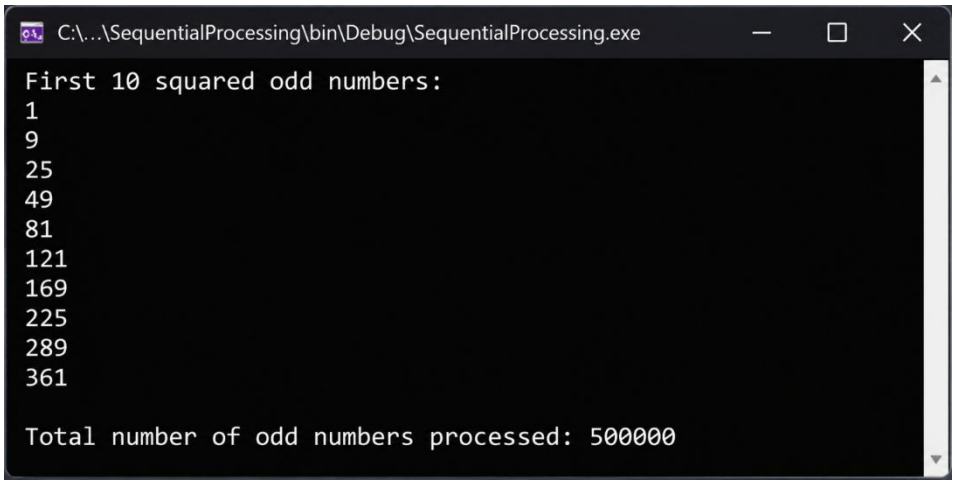
      foreach (long number in squaredOddNumbers.Take(10))
      {
```

```
            Console.WriteLine(number);
        }

        Console.WriteLine();

        Console.WriteLine("Total number of odd numbers processed: "
            + squaredOddNumbers.Count);
    }
}
```

OUTPUT:



DEPARTMENT OF IT		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT:

Thus, a C# program that filters out even numbers from a list and squares the remaining odd numbers using sequential processing has been done and verified successfully.